

# Dataframe

# Data Structures

- vectors
- factors
- lists
- matrices
- arrays
- ***dataframes***

# Dataframe

The dataframe is the most “complex”, useful and powerful structure in R. Intuitively it is an excel sheet, while from R’s point of view it is a type of **list** with some properties/restrictions

- each element of the list is a **vector** with an **associated name** (aka a column)
- each list/column must have the same number of elements
- as a consequence, each row has the same number of elements (**rectangular structure**)

# Creation

Most of the time you will import dataframes rather than create them, but it is important to first understand how they work and how to create/manipulate them. They are created with the `data.frame` command

```
1 # Create a dataframe with 3 columns
2 my_df = data.frame( col1 = 1:4, col2 = letters[1:4],
3                     col3 = rnorm(n = 4, mean = 0, sd = 1))
4 my_df
```

|   | col1 | col2 | col3       |
|---|------|------|------------|
| 1 | 1    | a    | -1.2164427 |
| 2 | 2    | b    | 0.9578252  |
| 3 | 3    | c    | 1.2170114  |
| 4 | 4    | d    | 0.5070257  |

# Attributes

The **dataframe** has both the attributes of a list, i.e. the ***names***, and the attributes of a matrix, i.e. the ***dimensions*** (rows and columns)

```
1 typeof(my_df)
```

```
[1] "list"
```

```
1 attributes(my_df)
```

```
$names
```

```
[1] "col1" "col2" "col3"
```

```
$class
```

```
[1] "data.frame"
```

```
$row.names
```

```
[1] 1 2 3 4
```

```
1 dim(my_df)
```

```
[1] 4 3
```

As with the other structures we have seen, we can use the functions `names()` , `dim()`, `nrow()`, `ncol()`... to get information about the properties of the dataframe. The most useful function is `str()`, since it gives us a quick overview of the structure of the dataframe: dimensions, variable types,...

```
1 str(iris) # iris is a dataset available by default in R (?iris)
```

```
'data.frame':  150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
```

# Indexing

We can use either square brackets `[]`:

```
1 my_df[1] # extract a 4x1 data.frame
```

```
col1  
1    1  
2    2  
3    3  
4    4
```

```
1 my_df[[1]] # extract the first column of the data.frame
```

```
[1] 1 2 3 4
```

```
1 my_df[1,1] # extract the first element of the first column of the data.frame
```

```
[1] 1
```

or the dollar sign `$`

```
1 my_df$col1 # extract the first column of the data.frame
```

```
[1] 1 2 3 4
```

```
1 my_df$col1[1] # extract the first element of the first column of the data.frame
```

```
[1] 1
```

# Indexing - Relational operators

One of the most common operations you will deal with is extracting/evaluating a subset of the values in your dataset:

```
1 my_df
```

|   | col1 | col2 | col3       |
|---|------|------|------------|
| 1 | 1    | a    | -1.2164427 |
| 2 | 2    | b    | 0.9578252  |
| 3 | 3    | c    | 1.2170114  |
| 4 | 4    | d    | 0.5070257  |

```
1 # keep only the rows where the values of column1 are greater than 2
2 my_df[my_df$col1 > 2, ]
```

|   | col1 | col2 | col3      |
|---|------|------|-----------|
| 3 | 3    | c    | 1.2170114 |
| 4 | 4    | d    | 0.5070257 |

```
1 my_df[my_df[1] > 2, ]
```

|   | col1 | col2 | col3      |
|---|------|------|-----------|
| 3 | 3    | c    | 1.2170114 |
| 4 | 4    | d    | 0.5070257 |

# Examples

|   | col1 | col2 | col3       |
|---|------|------|------------|
| 1 | 1    | a    | -1.2164427 |
| 2 | 2    | b    | 0.9578252  |
| 3 | 3    | c    | 1.2170114  |
| 4 | 4    | d    | 0.5070257  |

```
1 my_df[my_df$col1 > 2 & my_df$col1 < 4, ]
```

|   | col1 | col2 | col3     |
|---|------|------|----------|
| 3 | 3    | c    | 1.217011 |

```
1 my_df[my_df$col1== 2, 2]
```

```
[1] "b"
```

```
1 my_df[my_df$col1== 2, "col2"]
```

```
[1] "b"
```

# Indexing

There are also alternative, more compact ways of indexing. For example, using the `subset()` function:

```
1 subset(iris, subset = Species == "setosa" & Petal.Length > 1.7)
```

|    | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|----|--------------|-------------|--------------|-------------|---------|
| 25 | 4.8          | 3.4         | 1.9          | 0.2         | setosa  |
| 45 | 5.1          | 3.8         | 1.9          | 0.4         | setosa  |

equivalent to:

```
1 iris[iris$Species == "setosa" & iris$Petal.Length > 1.7,]
```

|    | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|----|--------------|-------------|--------------|-------------|---------|
| 25 | 4.8          | 3.4         | 1.9          | 0.2         | setosa  |
| 45 | 5.1          | 3.8         | 1.9          | 0.4         | setosa  |

We can also select columns rather than rows with the *select* argument:

```
1 subset(iris, select = c(Sepal.Length, Species))
```

```
1 # show the first rows with the head command
```

```
2 head(subset(iris, select = c(Sepal.Length, Species)), n = 3)
```

|   | Sepal.Length | Species |
|---|--------------|---------|
| 1 | 5.1          | setosa  |
| 2 | 4.9          | setosa  |
| 3 | 4.7          | setosa  |

We can also combine the two:

```
1 subset(iris, subset = Species == "setosa" & Sepal.Length > 4, select = c(Sepal.Length, Species))
```

```
1 head(subset(iris, subset = Species == "setosa" & Sepal.Length > 4, select = c(Sepal.Length, Species)), n = 3)
```

|   | Sepal.Length | Species |
|---|--------------|---------|
| 1 | 5.1          | setosa  |
| 2 | 4.9          | setosa  |
| 3 | 4.7          | setosa  |

... equivalent to:

```
1 head(iris[iris$Species == "setosa" & iris$Sepal.Length > 4, c("Sepal.Length","Species")],n = 3)
```

|   | Sepal.Length | Species |
|---|--------------|---------|
| 1 | 5.1          | setosa  |
| 2 | 4.9          | setosa  |
| 3 | 4.7          | setosa  |

# Indexing

Most of the time you will access variables with the `$` operator. This command can also be used to create a new variable...

```
1 # create a variable that is the sum of Length and Width
2 iris$sum = iris$Sepal.Length + iris$Sepal.Width
3
4 str(iris)
```

```
'data.frame':  150 obs. of  6 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
 $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...
 $ sum          : num  8.6 7.9 7.9 7.7 8.6 9.3 8 8.4 7.3 8 ...
```

The same concepts we have seen for vectors apply, so you can both create and modify variables.

```
1 my_df = data.frame(num = 1:4, let = letters[1:4])
2 my_df
```

```
  num let
1    1  a
2    2  b
3    3  c
4    4  d
```

```
1 # Modify the num variable by adding 1
2 my_df$num = my_df$num+1
3
4 # Create a third variable made of the num and let variables
5 my_df$both = paste(my_df$num,my_df$let, sep = "_") # ?paste
6
7 str(my_df)
```

```
'data.frame':  4 obs. of  3 variables:
 $ num : num  2 3 4 5
 $ let : chr  "a" "b" "c" "d"
 $ both: chr  "2_a" "3_b" "4_c" "5_d"
```

# Combining Dataframes

Since they are similar to matrices, dataframes can be combined with the `rbind()` function:

```
1 # first dataframe
2 str(my_df)
```

```
'data.frame':  4 obs. of  3 variables:
 $ num : num  2 3 4 5
 $ let : chr  "a" "b" "c" "d"
 $ both: chr  "2_a" "3_b" "4_c" "5_d"
```

```
1 # create a second dataframe
2 my_df2 = data.frame(num = 4:7, lett = letters[1:4],
3                     both = paste(4:7, letters[1:4], sep = "_"))
4
5 str(my_df2)
```

```
'data.frame':  4 obs. of  3 variables:
 $ num : int  4 5 6 7
 $ lett: chr  "a" "b" "c" "d"
 $ both: chr  "4_a" "5_b" "6_c" "7_d"
```

# Join the two dataframes

- The dataframes must have the same number of columns
- The column names must be identical

```
1 my_df3 = rbind(my_df,my_df2)
```

```
Error in `match.names()`:  
! names do not match previous names
```

```
1 str(my_df)
```

```
'data.frame':  4 obs. of  3 variables:  
 $ num : num  2 3 4 5  
 $ let : chr  "a" "b" "c" "d"  
 $ both: chr  "2_a" "3_b" "4_c" "5_d"
```

```
1 str(my_df2)
```

```
'data.frame':  4 obs. of  3 variables:  
 $ num : int  4 5 6 7  
 $ lett: chr  "a" "b" "c" "d"  
 $ both: chr  "4_a" "5_b" "6_c" "7_d"
```

# Fix the names

```
1 names(my_df2)
```

```
[1] "num" "lett" "both"
```

```
1 names(my_df)
```

```
[1] "num" "let" "both"
```

```
1 # I want the names of my_df2 to match the names of my_df
2 names(my_df2) = names(my_df)
3
4 my_df3 = rbind(my_df,my_df2)
5 str(my_df3)
```

```
'data.frame':  8 obs. of  3 variables:
 $ num : num  2 3 4 5 4 5 6 7
 $ let : chr  "a" "b" "c" "d" ...
 $ both: chr  "2_a" "3_b" "4_c" "5_d" ...
```

You may also need to collect different types of data from the same participant, and then combine the collected information...

Dataframe containing the reaction times:

```
1 df_rt = data.frame(subj = factor(rep(c("alice","bob"),each = 400)),
2                       cond = factor(rep(c("easy","hard"),
3                                           each = 200, times = 2)),
4                       rt = c(rlnorm(n = 400, meanlog = -1, sdlog = .25),
5                             rlnorm(n = 400, meanlog = -.7, sdlog = .3)))
6
7 # equivalent to first creating the variables as vectors and then putting them in a df
8 subj = factor(rep(c("alice","bob"),each = 400))
9 cond = factor(rep(c("easy","hard"),each = 200, times = 2))
10 rt = c(rlnorm(n = 400, meanlog = -1, sdlog = .25),
11         rlnorm(n = 400, meanlog = -.7, sdlog = .3))
12
13 df_rt = data.frame(subj, cond, rt)
```

Dataframe containing the age:

```
1 df_age = data.frame(subj = factor(c("alice","bob")), age = c(20,3))
```

```
1 str(df_rt) # structure of the reaction times dataframe
```

```
'data.frame':  800 obs. of  3 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 ...
 $ rt  : num  0.572 0.391 0.268 0.329 0.405 ...
```

```
1 head(df_rt)
```

```
  subj cond      rt
1 alice easy 0.5724112
2 alice easy 0.3905852
3 alice easy 0.2675803
4 alice easy 0.3291322
5 alice easy 0.4053720
6 alice easy 0.3111655
```

```
1 str(df_age) # structure of the age dataframe
```

```
'data.frame':  2 obs. of  2 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 2
 $ age : num  20 3
```

```
1 head(df_age)
```

```
  subj age
1 alice  20
2  bob   3
```

In this case, we can use the `merge()` function and/or the `_join()` functions:

```
1 df_all_1 = merge(x = df_rt, y = df_age, by="subj")
2 str(df_all_1)
```

```
'data.frame':  800 obs. of  4 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 ...
 $ rt  : num  0.572 0.391 0.268 0.329 0.405 ...
 $ age : num  20 20 20 20 20 20 20 20 20 20 ...
```

```
1 library(dplyr) # load the dplyr package
2
3 df_all_2 = left_join(x = df_rt, y = df_age, by = c("subj")) # right_join and full_join also exist
4 str(df_all_2)
```

```
'data.frame':  800 obs. of  4 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 ...
 $ rt  : num  0.572 0.391 0.268 0.329 0.405 ...
 $ age : num  20 20 20 20 20 20 20 20 20 20 ...
```

# Contingency tables

We can use the `table()` function both to compute frequencies and to compute contingency tables, for example to see how many trials we have for each subject.

```
1 # keep only rts between .250 and 1.5 seconds
2 # i.e. the rows that meet this condition
3 df_rt_clean = df_rt[df_rt$rt > .25 & df_rt$rt < 1.5, ]
4
5 # how many trials for each subject
6 table(df_rt$subj)
```

```
alice  bob
400    400
```

```
1 table(df_rt_clean$subj)
```

```
alice  bob
369    396
```

```
1 # how many trials for each subject x condition
2 table(df_rt_clean$subj, df_rt_clean$cond)
```

```
      easy hard
alice 186  183
bob   200  196
```

# Exporting and importing data

In R we can import data in many different formats; most commonly you will import **.csv** or **.xlsx** data.

Here, for example, I export the dataframes in two different formats...

```
1 library(readr) # load the readr package
2 library(writexl) # load the writexl package
3
4 write_csv(df_rt, file = "DATA/df_rt.csv", row.names = FALSE)
5 write_xlsx(df_age, path = "DATA/df_age.xlsx")
```

# Importing

```
1 library(readxl) # load the readxl package
2
3 df_rt_impo = read_csv("DATA/df_rt.csv") # uses the readr package
4
5 df_age_impo = read_xlsx("DATA/df_age.xlsx")
```

# Checking the structure

What differences do you notice between the two dataframes?

```
1 str(df_rt) # Data frame we created
```

```
'data.frame':  800 obs. of  3 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 ...
 $ rt  : num  0.572 0.391 0.268 0.329 0.405 ...
```

```
1 str(df_rt_impo) # Data frame we loaded
```

```
spc_tbl_ [800 × 3] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ subj: chr [1:800] "alice" "alice" "alice" "alice" ...
 $ cond: chr [1:800] "easy" "easy" "easy" "easy" ...
 $ rt  : num [1:800] 0.572 0.391 0.268 0.329 0.405 ...
- attr(*, "spec")=
 .. cols(
 ..   subj = col_character(),
 ..   cond = col_character(),
 ..   rt = col_double()
 .. )
- attr(*, "problems")=<externalptr>
```

The dataframe is imported as a tibble, and in this case the *factor* variables have been imported as *character*. This can be changed either when importing ( ? *read\_csv* ):

```
1 df_rt_imp01 = read_csv("DATA/df_rt.csv",
2                         col_types =
3                         list(col_factor(),col_factor(),
4                             col_double()))
5 str(df_rt_imp01)
```

```
spc_tbl_ [800 × 3] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 ...
 $ rt   : num [1:800] 0.572 0.391 0.268 0.329 0.405 ...
- attr(*, "spec")=
 .. cols(
 ..   subj = col_factor(levels = NULL, ordered = FALSE, include_na = FALSE),
 ..   cond = col_factor(levels = NULL, ordered = FALSE, include_na = FALSE),
 ..   rt = col_double()
 .. )
- attr(*, "problems")=<externalptr>
```

or afterwards with the `as.factor()` command:

```
1 df_rt_impo$subj = as.factor(df_rt_impo$subj)
2 df_rt_impo$cond = as.factor(df_rt_impo$cond)
3 str(df_rt_impo)
```

```
spc_tbl_ [800 × 3] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 1 ...
 $ rt   : num [1:800] 0.572 0.391 0.268 0.329 0.405 ...
- attr(*, "spec")=
 .. cols(
 ..   subj = col_character(),
 ..   cond = col_character(),
 ..   rt = col_double()
 .. )
- attr(*, "problems")=<externalptr>
```

equivalent to:

```
1 df_rt_impo2 = read_csv("DATA/df_rt.csv") %>% #library(tidyverse)
2   mutate(subj = factor(subj),
3          cond = factor(cond))
4 str(df_rt_impo2)
```

```
tibble [800 × 3] (S3: tbl_df/tbl/data.frame)
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1 1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1 1 ...
 $ rt   : num [1:800] 0.572 0.391 0.268 0.329 0.405 ...
```

(just to show you that in **R** a problem can be solved in different ways)

## The same concepts apply to reading xlsx files:

```
1 str(df_age_impo)
```

```
tibble [2 × 2] (S3: tbl_df/tbl/data.frame)
 $ subj: chr [1:2] "alice" "bob"
 $ age : num [1:2] 20 3
```

```
1 df_age_impo$subj=as.factor(df_age_impo$subj)
2 str(df_age_impo)
```

```
tibble [2 × 2] (S3: tbl_df/tbl/data.frame)
 $ subj: Factor w/ 2 levels "alice","bob": 1 2
 $ age : num [1:2] 20 3
```

```
1 df_age_impo$subj=factor(df_age_impo$subj)
2 str(df_age_impo)
```

```
tibble [2 × 2] (S3: tbl_df/tbl/data.frame)
 $ subj: Factor w/ 2 levels "alice","bob": 1 2
 $ age : num [1:2] 20 3
```

The examples I have shown concern data in formats **external** to R, but we can also save and import data in **R** formats.

Their main drawback is that they can be read (mainly) only within the R environment.

If we need to work with different software or languages, it is probably better to use a more generic format.

If we have time left, we will also look at this type of format; the material is available at this [link](#) (in Italian).

# Now let's practice!

Open this link and keep it open:

<https://etherpad.wikimedia.org/p/arca-corsoR>